DSP MINI PROJECT REPORT

ON

" Comparative DSP Benchmark: DFT, FFT, FIR and IIR "

Under the Course

DIGITAL SIGNAL PROCESSING

SUBMITTED BY

PRN	NAME OF STUDENT
1022512013	Raghav Bholeshwar Palange

2026-27

Third Year Electronics and Communication Engineering (TY ECE)

Division:B , Semester - I

Under the Guidance Of
Dr. Hrishikesh Vanjari

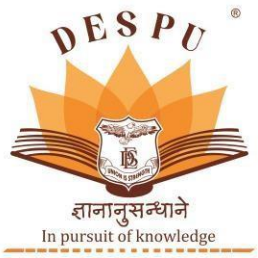
Department of Electronics and Communication
School of Engineering and Technology
DES Pune University, Pune.

2026-2027

Program: B. Tech Electronics and Communication (AI/ML)Engineering

School of Engineering and Technology

DES Pune University, Pune



CERTIFICATE

This is to certify that student **Raghav Bholshwar Palange** , PRN.**1022512013** is studying in **TY ECE (Division:B)** course in SEM I and he has successfully completed and submitted the **DSP Mini Report** under the Course **"Digital Signal Processing"** , entitled **"Comparative DSP Benchmark: DFT, FFT, FIR and IIR"** .This study is a partial fulfillment of the degree of Bachelor of Technology in Computer Science and Engineering of **DES Pune University, Pune** during the academic year **2026-2027.**

Dr. Hrishikesh Vanjari — Mentor

Dr. Prachi Joshi — HoS

Acknowledgement

We would like to express our sincere gratitude to our faculty mentor, Dr. Hrishikesh Vanjari, for his valuable guidance, support, and encouragement throughout the completion of this Digital Signal Processing mini-project titled “Comparative DSP Benchmark: DFT, FFT, FIR and IIR.”

We would also like to thank the Department of Computer Engineering and Technology, School of Engineering and Technology, and DES Pune University, Pune, for providing the opportunity and academic environment to undertake this project.

We are grateful to the developers of MATLAB and its documentation, and to the authors of the research papers and the SciPy library listed in the references, for providing the tools and technical resources required to implement the DFT, FFT, convolution, FIR and IIR filters and to evaluate the experimental results using runtime, RMSE and operation counts.

This project has helped us strengthen our understanding of spectral analysis, digital filtering, computational complexity, and the practical application of Digital Signal Processing concepts.

Abstract

Digital signal processing relies on a small set of core algorithms: the discrete Fourier transform (DFT), the fast Fourier transform (FFT), convolution, and finite and infinite impulse response (FIR and IIR) filters. Each has a trade-off between accuracy and computation, but they are usually studied separately. This project builds one reproducible MATLAB benchmark in which all of them are tested on the same signals under the same protocol.

The benchmark uses synthetic signals (sinusoids with noise) and a real audio recording (MATLAB's built-in handel signal). We implemented a naive DFT, our own radix-2 FFT, direct and FFT-based convolution, a 50th-order Hamming-window FIR filter and a 4th-order Butterworth IIR filter, and compared them with MATLAB's built-in functions, which serve as the existing method. The metrics are runtime, RMSE, frequency resolution, operation count, stopband attenuation and passband ripple.

Our FFT was 19.2 times faster than the naive DFT at N = 4096 and matched the built-in FFT to about 1e-14. FFT-based convolution was 6.7 times faster than direct convolution at length 32768. The FIR filter gave 60.7 dB of stopband attenuation against 16.5 dB for the IIR, with linear phase, while the zero-phase IIR reached the same RMSE (about 0.094) with 34 operations per sample against 61 for the FIR. The code, tables and figures form a reusable benchmark framework.

Keywords: DFT, FFT, FIR filter, IIR filter, convolution, benchmark, MATLAB, RMSE, computational complexity

INDEX

Chapter No.	Topic	Page No.
1	**Introduction**	**7**
1.1	Background	7
1.2	Need/Importance	7
1.3	Objectives	7
1.4	Scope	7
2	**Literature Review**	**8**
2.1	Relevant concepts, theories	8
2.2	Technologies related to the project	9
2.3	Summary of similar works or case studies	9
3	**Proposed Methodology**	**10**
3.1	System Overview	10
3.2	Architecture / Workflow Diagram	10
3.3	Data Collection / Input Description	10
3.4	Algorithms / Techniques Used	11
3.5	Tools, software, or equipment used	11
3.6	Module-wise Implementation Plan	11

Chapter No.	Topic	Page No.
3.7	Validation and Testing Strategy	12
4	Results / Observations	13
4.1	Experimental / Implementation Setup	13
4.2	Performance Metrics	13
4.3	Comparison with Expected Results	13
4.4	Observations and Analysis	15
4.5	Validation and Verification with summary of findings	23
5	**Future Enhancement**	**25**
6	**Conclusion & Recommendations**	**26**
7	**References / Bibliography**	**27**

1. Introduction

1.1 Background

Most modern systems, such as audio, communications, radar, biomedical monitoring and image processing, are built on a handful of DSP building blocks. The DFT converts a signal to the frequency domain, and the FFT computes it efficiently. Convolution and digital filters (FIR and IIR) are used to remove noise and to shape the spectrum.

1.2 Need/Importance

These algorithms are normally compared by theory alone, and there is no single consistent test that measures their accuracy and computational cost on common signals. An engineer choosing between a DFT and an FFT, or between an FIR and an IIR filter, needs numbers for runtime, error and operation count. This project provides them with one reproducible benchmark, which matters most for real-time and embedded systems where computation is limited.

1.3 Objectives

1. Implement the DFT, FFT, convolution, FIR and IIR filters in MATLAB.
2. Evaluate them on synthetic signals and a real dataset using one consistent protocol.
3. Measure runtime, RMSE, frequency resolution and operation count.
4. Compare our implementations with MATLAB's built-in functions (the existing method).
5. Produce a reusable framework and paper-ready tables and graphs.

1.4 Scope

The project covers one-dimensional discrete-time signals, a lowpass filtering task, and software implementation in MATLAB. Hardware implementation, adaptive filtering and multirate filter banks are outside the scope and are listed under future enhancement.

2. Literature Review and Theory

2.1 Relevant concepts, theories

DFT. For a sequence x[n] of length N, the DFT is defined below. Computing it directly needs N^2 complex multiplications.

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j 2\pi k n / N}, \quad k = 0,1,\dots,N-1$$

Frequency resolution. The spacing between DFT bins depends on the sampling frequency and the number of samples. Zero-padding interpolates the spectrum but does not improve resolution; only a longer observation does.

$$\Delta f = f_s / N$$

FFT. Cooley and Tukey (1965) showed that the DFT can be split into two half-length DFTs of the even-indexed and odd-indexed samples. Repeating the split reduces the cost to $(N/2)\cdot\log_2 N$ complex multiplications. We use the radix-2 decimation-in-time form, which needs N to be a power of 2.

Convolution. Linear convolution of a signal of length L with a filter of length M costs about $L\cdot M$ operations in direct form. By the convolution theorem, multiplying the FFTs of the zero-padded signals and taking the inverse FFT gives the same result at a cost proportional to $N\cdot\log N$.

FIR filters. The output is a weighted sum of current and past inputs only. FIR filters are always stable and can have exactly linear phase (constant delay, no waveform distortion), but a sharp transition needs a high order. We use the window method (fir1 with a Hamming window).

IIR filters. The output also depends on past outputs (feedback). An IIR filter gets a sharp response with a low order, but its phase is non-linear and stability must be checked. We use a Butterworth design, which has a maximally flat passband.

RMSE. The error between a processed signal y and a reference r over M samples is measured as follows.

$$\mathrm{RMSE} = \sqrt{\frac{1}{M}\sum_{m=1}^{M}\left(y[m]-r[m]\right)^2}$$

2.2 Technologies related to the project

1. **MATLAB** with the Signal Processing Toolbox: fft, conv, fir1, butter, freqz, grpdelay, filter and filtfilt.
2. **timeit:** MATLAB's function for robust timing of code, which repeats the call and reports a stable estimate.
3. **SciPy (Python):** the open-source equivalent of these functions (scipy.fft and scipy.signal), used as a reference for the algorithms and benchmarking practice.
4. **handel.mat:** an audio recording supplied with MATLAB, used as the real dataset.

2.3 Summary of similar works or case studies

- **Cooley and Tukey (1965)** introduced the FFT algorithm that this project implements and measures.
- **Agrawal and Sahu (2013)** give an overview of two-channel quadrature mirror filter (QMF) banks, in which FIR and IIR designs are compared for near-perfect reconstruction. This motivates a fair FIR versus IIR comparison.
- **The SciPy paper (arXiv:2502.14224)** describes the open-source signal-processing library, a reference implementation of these algorithms.
- **Standard textbooks** (Oppenheim and Schafer; Proakis and Manolakis) describe each algorithm and its theoretical complexity.

These works describe the algorithms and their theory. From the resources we studied, none gives one unified, reproducible measurement of DFT, FFT, convolution, FIR and IIR on the same signals, which is the gap this project fills.

3. Methodology

3.1 System Overview

The system is a single MATLAB script (dsp_benchmark.m) that generates the test signals, runs every algorithm under a common protocol, measures runtime, error, operation count and frequency response, and saves four CSV tables and seven PNG figures. A fixed random seed (rng(1)) makes every run reproducible.

3.2 Architecture / Workflow Diagram

```
Input signals (synthetic + real handel audio, fixed random seed)
                    |
                    v
Algorithms under test
[Naive DFT] [Own FFT] [Direct conv] [FFT conv] [FIR] [IIR]
     compared with MATLAB built-ins (existing method)
                    |
                    v
Measurements: runtime | RMSE | operations | resolution | frequency response
                    |
                    v
Tables 1-4 (CSV)  and  Figures 1-7 (PNG)
                    |
                    v
Analysis, comparison with existing method, conclusions
```

3.3 Data Collection / Input Description

1. **Signal A (synthetic, DFT/FFT):** 50 Hz and 120 Hz sinusoids plus Gaussian noise (standard deviation 0.1), fs = 1000 Hz, N from 64 to 4096.
2. **Signal B (resolution test):** two tones at 100 Hz and 105 Hz, fs = 1000 Hz.
3. **Signal C (filtering, synthetic):** wanted tones at 50 Hz and 80 Hz, plus an unwanted 300 Hz tone and white noise (standard deviation 0.2), fs = 1000 Hz, duration 2 s.
4. **Signal D (real dataset):** first 32768 samples of MATLAB's handel audio (fs = 8192 Hz), normalised, with added noise of standard deviation 0.05.

The synthetic signals are generated by the code itself, so no external download is needed.

3.4 Algorithms / Techniques Used

Category	Our implementation	Existing method (reference)
Transform	Naive DFT (direct from definition); own recursive radix-2 FFT	MATLAB fft
Convolution	Direct loop; FFT-based with zero-padding	MATLAB conv
Filtering	FIR: order 50, Hamming window, cut-off 120 Hz; IIR: Butterworth order 4, cut-off 120 Hz (1000 Hz for audio)	MATLAB filter and filtfilt

The FIR output is shifted back by its constant delay of order/2 samples before the error is computed. The IIR is evaluated both causally (filter) and with zero phase (filt), because comparing a delayed output with an undelayed reference would be unfair.

3.5 Tools, software, or equipment used

MATLAB [version] with the Signal Processing Toolbox, running on [processor, RAM, operating system]. Code is kept on GitHub: [link]. Figures and tables are produced automatically by the script.

3.6 Module-wise Implementation Plan

Module	Content	Output
1	DFT vs FFT benchmark (dft_naive, myfft)	Table 1, Fig. 1, Fig. 2
2	Frequency resolution and zero-padding	Fig. 3
3	Convolution benchmark (conv_direct, conv_fft)	Table 2, Fig. 4

Module	Content	Output
4	FIR vs IIR on synthetic and real signals (eval_filters)	Table 3, Fig. 5, Fig. 6
5	Accuracy versus computation order sweep	Table 4, Fig. 7

3.7 Validation and Testing Strategy

- Every implementation is checked against the corresponding MATLAB built-in function (RMSE against fft and conv).
- Filters are tested on two different signals (synthetic and real) with the same protocol.
- Timing uses timeit, which repeats each call; start-up transients (first 200 samples) are excluded from filter errors.
- A fixed random seed makes noise, and hence all tables, reproducible.
- Theoretical predictions (operation counts, resolution, phase behaviour) are compared with the measured results in Section 4.3.

4. Results and observation

4.1 Experimental / Implementation Setup

All experiments were run by one script on MATLAB [version], on [processor, RAM, operating system]. Transform lengths were N = 64 to 4096 (powers of 2), and convolution lengths L = 256 to 32768 with a 255-tap filter. The order sweep used FIR orders 10 to 120 and IIR orders 2 to 8. Results are saved in the results folder as four CSV tables and seven PNG figures.

4.2 Performance Metrics

- **Runtime:** measured with timeit, in milliseconds.
- **RMSE:** for the transforms, against the built-in fft; for the filters, against the clean (noise-free) signal.
- **Frequency resolution:** $\Delta f = fs/N$.
- **Operation count:** DFT = N^2 and FFT = $(N/2)\log_2 N$ complex multiplications; FIR = 2·order + 1 and IIR = 4·order + 1 operations per output sample (zero-phase IIR counts double because it filters twice).
- **Stopband attenuation and passband ripple:** read from the frequency response (freqz).

4.3 Comparison with Expected Results

Aspect	Expected (theory)	Observed
DFT vs FFT cost	N^2 vs $(N/2)\log_2 N$, so speedup grows with N	Speedup 1.19 times at N = 64 and 19.2 times at N = 4096
FFT accuracy	Same as DFT up to rounding error	RMSE up to 2.6e-14 (FFT) against up to 4.1e-11 (DFT)
Resolution	$\Delta f = fs/N$	N = 128 (7.8 Hz) merges the 5 Hz-apart tones; N = 1024 (0.98 Hz) separates them

Aspect	Expected (theory)	Observed
FFT convolution	Faster than direct for long signals	6.7 times faster at L = 32768, error about 1e-16
FIR phase	Linear (constant delay)	Constant group delay (Fig. 5)
FIR vs IIR stopband	FIR needs more taps for the same sharpness	Stopband −60.7 dB (FIR) against −16.5 dB (IIR)
IIR cost	Fewer operations	17 against 101 operations per sample
IIR phase	Non-linear, distorts waveform	Causal RMSE 0.917 against 0.094 with zero-phase filtering

4.4 Observations and Analysis

4.4.1 DFT versus FFT

Table 1: DFT vs FFT (times in milliseconds)

N	DFT_ms	MyFFT_ms	BuiltinFFT_ms	RMSE_DFT	RMSE_MyFFT	Ops_DFT	Ops_FFT	Speed up
64	0.13342	0.1121	0.00029343	7.0919e-14	1.2782e-15	4096	192	1.19
128	0.48776	0.23559	0.00041031	1.7189e-13	2.7813e-15	16384	448	2.07
256	2.2307	0.46378	0.00070402	4.0442e-13	4.7474e-15	65536	1024	4.81
512	5.8507	0.98172	0.0011412	1.4858e-12	6.9746e-15	262144	2304	5.96
1024	16.673	2.0607	0.0020034	4.232e-12	1.1483e-14	1048576	5120	8.09
2048	48.481	4.2867	0.0036085	1.491e-11	1.8364e-14	4194304	11264	11.31
4096	153.6	7.9877	0.0076843	4.1035e-11	2.5772e-14	16777216	24576	19.23

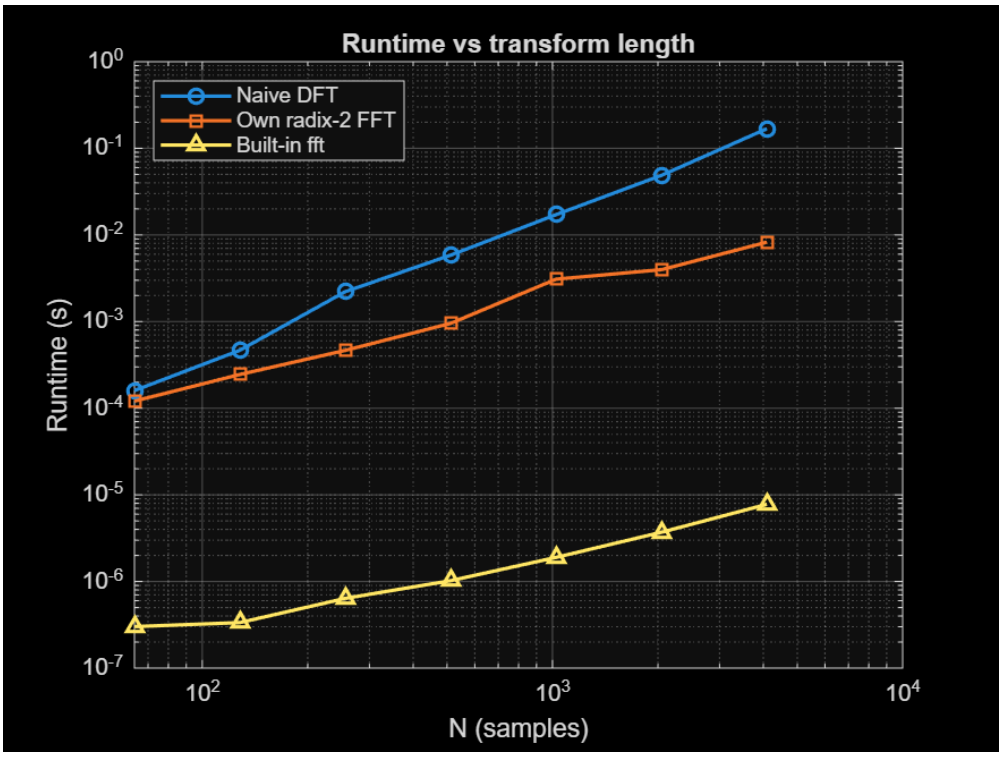


Fig. 1: Runtime versus transform length N (log-log scale).

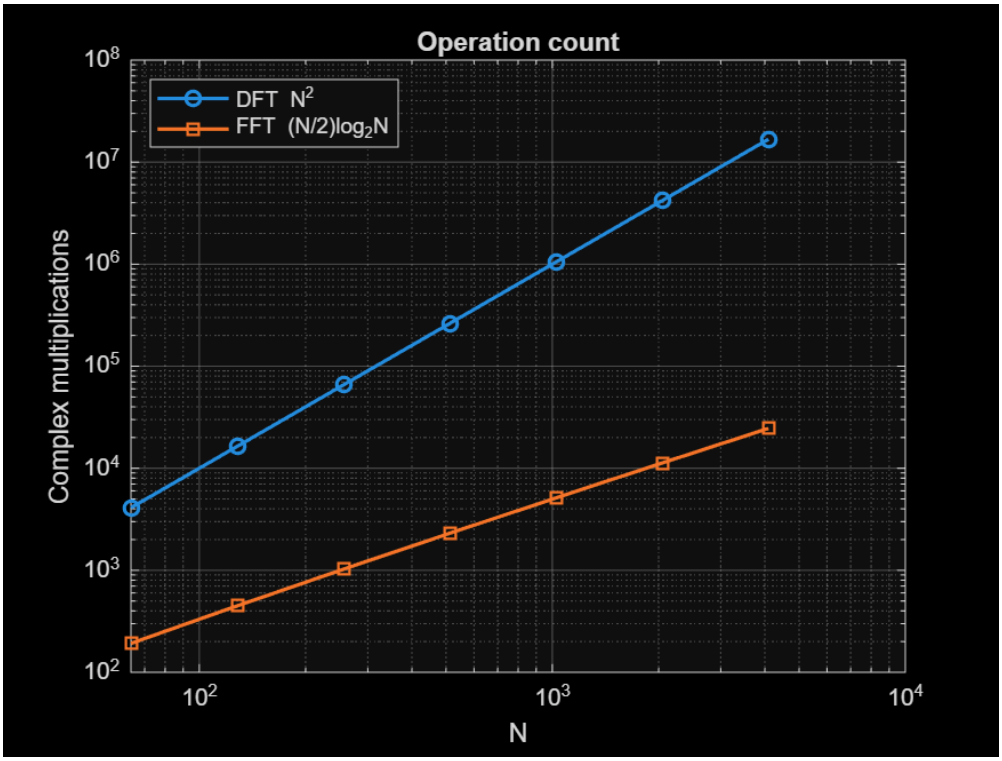


Fig. 2: Number of complex multiplications for the DFT and FFT.

Analysis. The speedup of our FFT over the naive DFT grows from 1.19 times at N = 64 to 19.2 times at N = 4096, as the operation counts predict. Both agree with the built-in FFT to within 1e-11, so the FFT is an exact fast algorithm, not an approximation. The DFT error grows with N because it accumulates more additions, while the FFT error stays about 1000 times smaller. The built-in FFT is far faster than ours because it is compiled code, whereas ours is interpreted as a recursive MATLAB. For N of 256 or less the built-in timings are near the timer's resolution and are approximate (MATLAB's timeit warned of this).

4.4.2 Frequency resolution

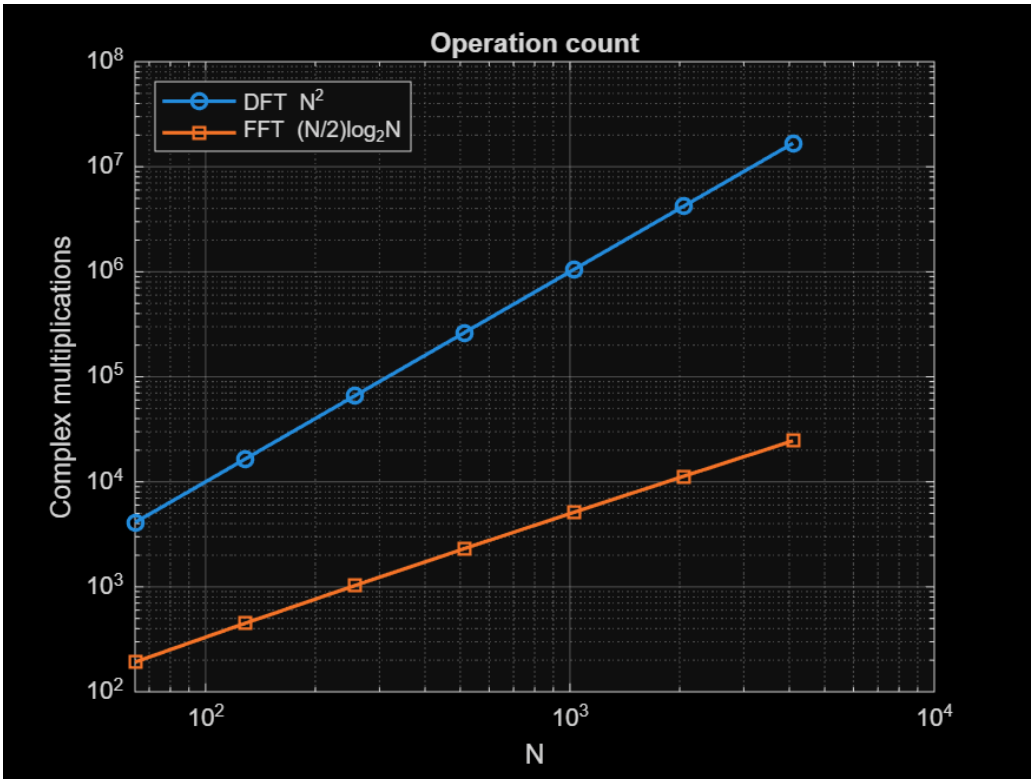


Fig. 3: Two tones at 100 Hz and 105 Hz analysed with N = 128, N = 128 zero-padded to 1024, and N = 1024.

Analysis. With N = 128 the resolution is 7.8 Hz, larger than the 5 Hz separation, so the tones merge into one peak. Zero-padding to 1024 gives a smoother curve but still does not separate them. With N = 1024 real samples the resolution is 0.98 Hz and two peaks are visible. Resolution depends on the observation length, and zero-padding only interpolates.

4.4.3 Convolution

Table 2: Convolution with a 255-tap filter (times in milliseconds)

L	DirectLoop_ms	FFTconv_ms	BuiltinConv_ms	RMSE_FFTconv
256	0.14908	0.016576	0.0041136	1.4148e-16
512	0.18796	0.01849	0.0072954	2.0654e-16
1024	0.24328	0.029711	0.014263	2.115e-16
2048	0.39676	0.051537	0.026277	2.3796e-16
4096	0.81826	0.099296	0.035594	2.5133e-16
8192	1.5537	0.24528	0.057537	2.458e-16
16384	3.0437	0.46544	0.074441	2.5822e-16
32768	10.568	1.5807	0.13567	2.6075e-16

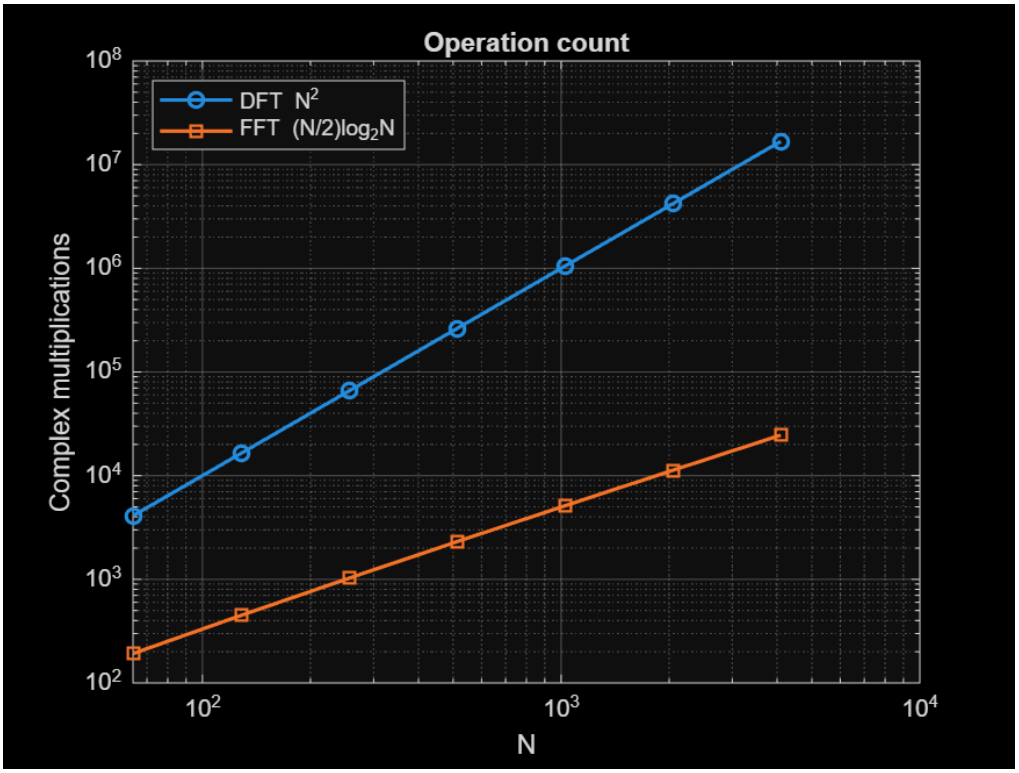


Fig. 4: Convolution runtime versus signal length.

Analysis. FFT-based convolution was faster than the direct loop at every tested length, reaching 6.7 times at L = 32768. Its error against the built-in conv is about 1e-16, which is machine precision. The built-in conv is the fastest of all because it is optimized compiled code.

4.4.4 FIR versus IIR

Table 3: FIR vs IIR on synthetic and real signals

Signal	Filter	RMSE	Runtime_ms	Ops/sample	Stopband max (dB)	Passband ripple (dB)	Linear phase
Synthetic	FIR-50 Hamming	0.096378	0.010043	101	-60.658	0.36536	Yes
Synthetic	IIR Butter-4 (causal)	0.91742	0.013236	17	-16.503	0.58638	No
Synthetic	IIR Butter-4 (filtfilt)	0.094416	0.090047	34	-16.503	0.58638	Yes (offline)
Handel audio	FIR-50 Hamming	0.14232	0.04625	101	-57.716	0.34014	Yes
Handel audio	IIR Butter-4 (causal)	0.34076	0.20679	17	-16.578	0.58603	No
Handel audio	IIR Butter-	0.12865	0.70426	34	-16.578	0.58603	Yes (offline)

Signal	Filter	RMSE	Runtime_ms	Ops/sample	Stopband max (dB)	Passband ripple (dB)	Linear phase
	4 (filtfilt)						

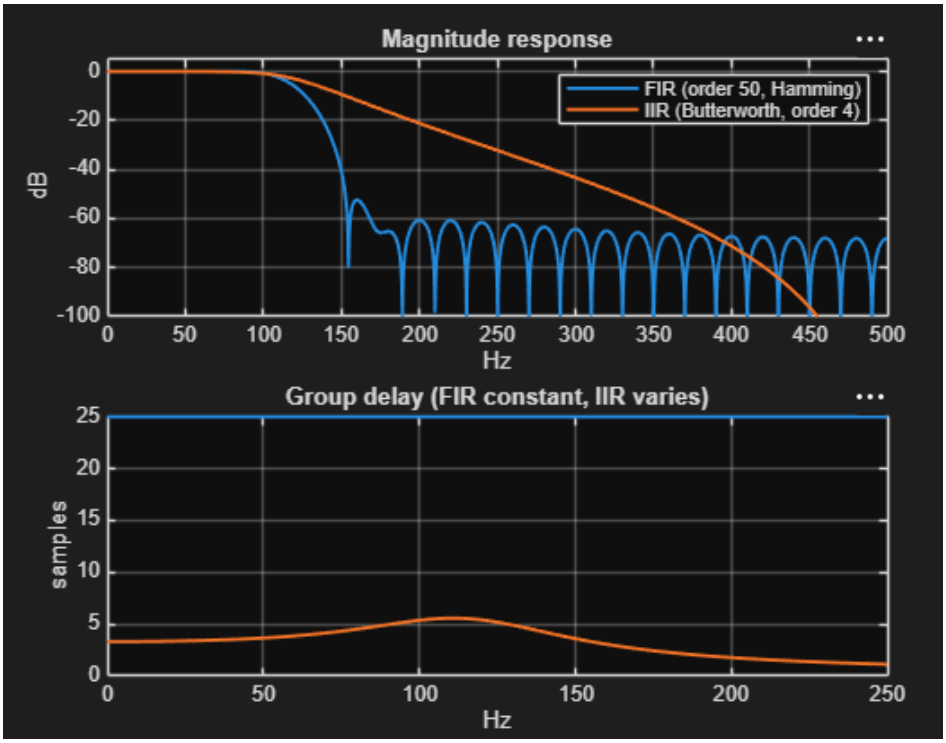


Fig. 5: Magnitude response (top) and group delay (bottom) of the FIR and IIR filters.

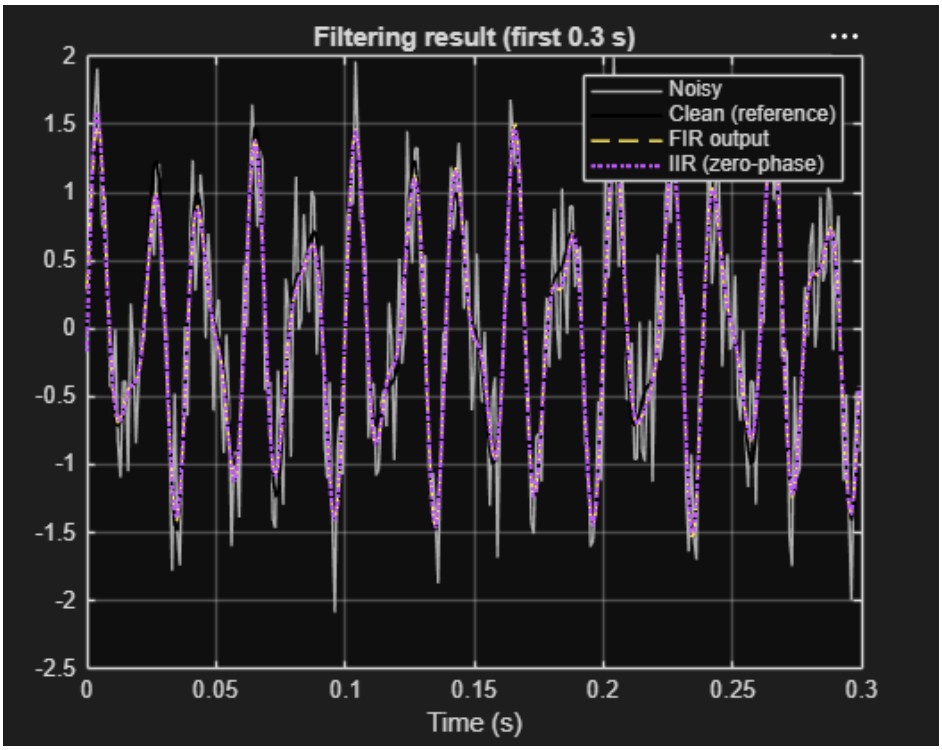


Fig. 6: Noisy, clean and filtered signals (first 0.3 s).

Analysis. The FIR has 60.7 dB of stopband attenuation against 16.5 dB for the IIR and constant group delay, but it costs 101 operations per sample against 17. The causal IIR has a large RMSE (0.917) because its non-linear phase distorts the waveform. With phase corrected by filtfilt, the IIR reaches RMSE 0.094, essentially equal to the FIR's 0.096. The same ordering appears on the real audio signal, so the conclusion does not depend on the synthetic signal. The stopband was measured at 1.5 times the cut-off (180 Hz), where a Butterworth filter has only begun to roll off; the unwanted tone is at 300 Hz, where the IIR attenuates much more, which is why both filters give a similar RMSE. The error does not reach zero because white noise inside the passband cannot be removed by any of these filters. Filter runtimes are around 0.01 ms and dominated by call overhead, so operations per sample is used as the main cost measure.

4.4.5 Accuracy versus computation

Table 4: Order sweep on the synthetic signal (IIR evaluated zero-phase)

Type	Order	RMSE	Runtime_ms	Ops/sample
FIR	10	0.13363	0.0034093	21
FIR	20	0.10243	0.007724	41
FIR	30	0.094408	0.0090354	61
FIR	50	0.096378	0.010053	101
FIR	80	0.097435	0.011482	161
FIR	120	0.097882	0.014191	241
IIR	2	0.10459	0.087888	18
IIR	3	0.094782	0.091881	26
IIR	4	0.093969	0.092615	34
IIR	5	0.09466	0.090964	42
IIR	6	0.095394	0.090979	50
IIR	8	0.096393	0.1015	66

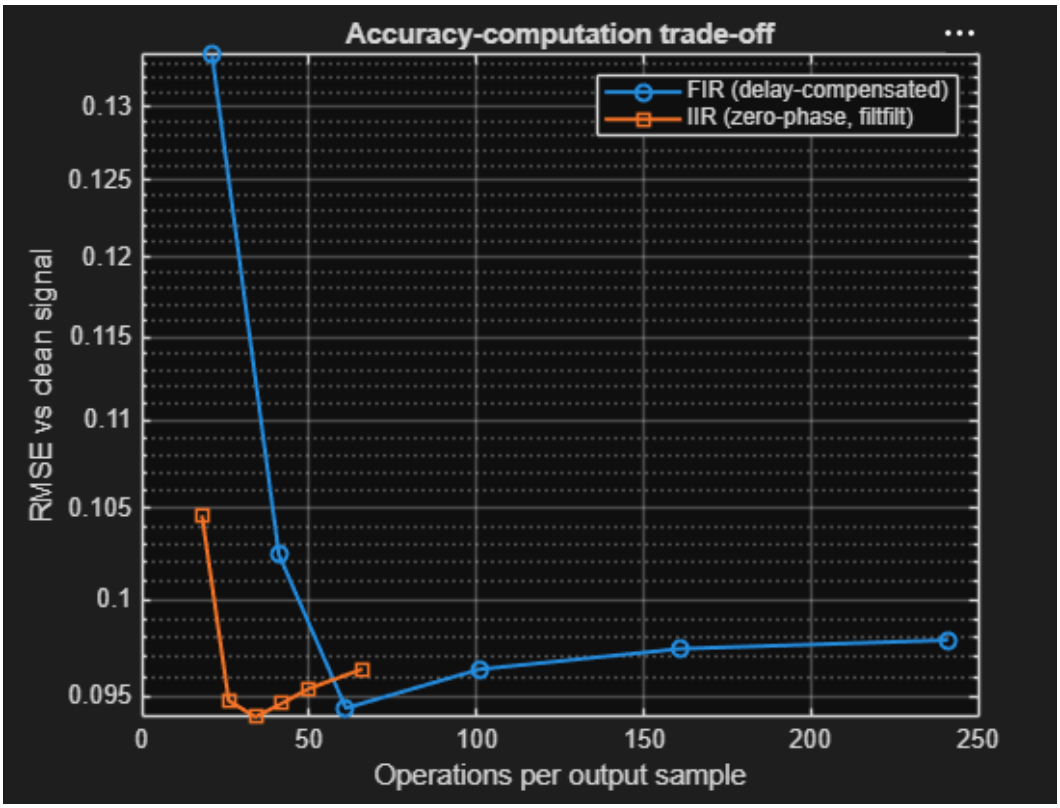


Fig. 7: RMSE versus operations per output sample for FIR and IIR filters.

Analysis. For both filters the RMSE falls quickly with order and then levels off near 0.094 because of the in-band noise floor. The FIR reaches this level at order 30 (61 operations per sample), and a higher order only adds cost. The IIR reaches it at order 4 with 34 operations per sample, so it is cheaper for the same accuracy. The differences in the plateau region (0.094 to 0.098) are small and should not be over-interpreted.

4.5 Validation and Verification with summary of findings

Validation against the existing method (MATLAB built-ins):

Item	Our implementation	Existing method	Verification result
FFT	myfft, 7.99 ms at N = 4096	fft, 0.0077 ms	RMSE about 1e-14; built-in about 1000 times faster (compiled code)

Item	Our implementation	Existing method	Verification result
Convolution	conv_fft, 1.58 ms at L = 32768	conv, 0.136 ms	RMSE about 1e-16; built-in faster
Filtering	fir1 and butter designs	filter, filtfilt	Built-in functions used for design and filtering

Summary of findings:

1. The FFT is up to 19.2 times faster than the DFT at N = 4096 with no loss of accuracy.
2. Frequency resolution depends on the number of real samples; zero-padding does not improve it.
3. FFT-based convolution is 6.7 times faster than direct convolution for long signals.
4. The FIR gives much better stopband attenuation and linear phase, at a higher cost per sample.
5. The IIR matches the FIR's accuracy at lower cost once its phase is compensated.
6. Our first test of the IIR without phase compensation made it look much worse (RMSE 0.92). That measurement was unfair, which shows why a consistent protocol is needed.
7. Limitations: results come from one computer, one filter design of each kind and one real dataset; runtime depends on hardware and MATLAB's optimisation.

CHAPTER 5: FUTURE ENHANCEMENT

- Add a two-channel QMF filter bank and test perfect reconstruction, following Agrawal and Sahu (2013).
- Add other designs (Chebyshev, elliptic, Kaiser-window FIR, Parks-McClellan) and other FFT variants (split-radix, Bluestein).
- Compare directly with Python/SciPy and with more real datasets such as ECG or speech.
- Measure on embedded hardware and add fixed-point arithmetic effects.
- Build a simple graphical interface so that users can choose a signal and algorithm and see the metrics.

CHAPTER 6: CONCLUSION & RECOMMENDATIONS

Conclusion

We built a reproducible MATLAB benchmark for the DFT, FFT, convolution, FIR and IIR filters using synthetic and real signals. The FFT was up to 19.2 times faster than the DFT at N = 4096 with no loss of accuracy, and FFT-based convolution was 6.7 times faster than direct convolution for long signals. The FIR filter offered much better stopband attenuation and linear phase, while the IIR reached the same RMSE at lower cost per sample once its phase was compensated. The framework and tables can be reused for other signals and algorithms.

Recommendations

- Use the FFT for any transform longer than a few hundred samples, and FFT-based convolution for long signals with long filters.
- Use an FIR filter when linear phase and a steep, well-controlled stopband are needed.
- Use an IIR filter when computation is limited, and apply it with zero phase (filtfilt) when processing offline.
- Always compare algorithms under one protocol, with delay compensation and a reference method.

CHAPTER 7: REFERENCES / BIBLIOGRAPHY

1. S. Agrawal and R. Sahu, “Two-Channel Quadrature Mirror Filter Bank: An Overview,” ISRN Signal Processing, 2013. https://doi.org/10.1155/2013/815619
2. SciPy open-source signal-processing library, arXiv:2502.14224. https://arxiv.org/abs/2502.14224
3. J. W. Cooley and J. W. Tukey, “An algorithm for the machine calculation of complex Fourier series,” Mathematics of Computation, vol. 19, pp. 297-301, 1965.
4. A. V. Oppenheim and R. W. Schafer, Discrete-Time Signal Processing, 3rd ed., Pearson, 2010.
5. J. G. Proakis and D. G. Manolakis, Digital Signal Processing: Principles, Algorithms and Applications, 4th ed., Pearson, 2007.
6. MathWorks, Signal Processing Toolbox documentation (fft, fir1, butter, freqz, filter, filt, timeit). https://www.mathworks.com/help/signal/
7. GitHub repository: https://github.com/raghavpalange2/dsp-benchmark-dft-fft-fir-iir